PROJECT DOCUMENTATION

Personalized Learning Recommendation System

A machine learning based system that recommends suitable topics and learning resources to students

Student Name	Aditya Paul
Student Code	BWU/BCA/24/083
Subject	Machine Learning
Subject Code	BCA57205
Department	Computational Science
Document	Project Report and Technical Documentation

Technologies: Python, Pandas, NumPy, Scikit-learn, Matplotlib, Streamlit

November 2026

Table of Contents

1	Introduction	3
2	Problem Statement and Objectives	4
3	Scope of the Project	5
4	Literature Review / Existing Systems	6
5	Proposed System	7
6	Project Methodology	8
7	System Architecture	9
8	Database Design and Data Flow	10
9	Dataset and Preprocessing	11
10	Exploratory Data Analysis	12
11	Recommendation Model and Feature Importance	13
12	Implementation	14
13	Testing and Results	15
14	Conclusion and Future Scope	16
15	References and Appendix	17

1. Introduction

Abstract

This project presents the Personalized Learning Recommendation System, a machine learning application that suggests suitable topics and learning resources to individual students. The system studies a student's previous performance, study hours, attendance, learning style, interests and learning history, and then ranks the topics that are most likely to be useful next. It combines content-based filtering, user-based collaborative filtering and K-Means clustering into a hybrid model, and it exposes the result through a simple Streamlit web interface.

Background

Digital learning platforms now offer thousands of courses, videos, articles and practice sets. Having a large choice does not automatically help a learner. A student who is strong in programming but weak in mathematics, who prefers video lectures over long articles, or who has only five hours a week to study, needs a different path from a classmate with another profile. Most platforms still show the same catalogue, or a list of the most popular items, to everybody.

Recommender systems are widely used in online shopping, music and video streaming to solve exactly this kind of information overload. The same ideas can be applied to education, where the items being recommended are topics, lessons and resources, and the signals are marks, time spent and completed activities.

Overview of the project

The project covers the full machine learning workflow, from preparing data to presenting results:

- Creating a student-learning dataset with students, topics and interaction records.
- Cleaning and preprocessing the data, followed by exploratory data analysis (EDA) and feature importance analysis.
- Building a hybrid recommendation model that also handles new students who have no history (the cold-start case).
- Evaluating the model against a popularity baseline using ranking metrics such as Precision@K, Recall@K, NDCG@K and HitRate@K.
- Delivering a working application where a user enters student information and receives personalised, explained recommendations.

The complete source code is organised as a GitHub-ready repository, so that the project can be reproduced by running two commands.

2. Problem Statement and Objectives

2.1 Problem statement

Students receive the same generic study material regardless of their performance, interests and pace of learning. When the material is too easy it becomes boring, when it is too hard it becomes discouraging, and when there is too much of it the student does not know where to start. Teachers cannot manually design a separate path for every learner in a large class, and online platforms rely mostly on popularity or keyword search.

The real-world issue addressed by this project is therefore: **how can a system automatically suggest the right topic or resource, to the right student, at the right time, using the data that is already available about the student?**

2.2 Challenges involved

- Students differ in ability, interest, study time and preferred type of resource.
- Interaction data is sparse: a student studies only a small fraction of all available topics.
- New students have no history, so a purely history-based model cannot serve them (cold start).
- Recommendations must be explainable so that students and teachers can trust them.

2.3 Objectives

- To select or create a suitable student-learning dataset.
- To perform data cleaning and preprocessing so that the data is reliable for modelling.
- To explore the dataset using EDA and identify the features that matter most for student learning.
- To develop a hybrid recommendation approach that combines content, collaborative and cluster information.
- To generate personalised Top-N topic recommendations together with a short reason for each.
- To evaluate the approach with standard ranking metrics and compare it with a baseline.
- To present the results with clear graphs and a working web application.

2.4 Expected outcome

A simple working system that takes student information as input and returns personalised learning topic recommendations, along with evaluation results that show whether the approach is better than recommending only the most popular topics.

3. Scope of the Project

3.1 Coverage

The project covers the end-to-end process of building a recommender for learning content: dataset creation, preprocessing, EDA, feature importance, model development, evaluation, and a user interface. The system recommends from a catalogue of 30 topics in six subjects (Mathematics, Programming, Data Science, Physics, Databases and Networking), each with five difficulty levels and a resource type (Video, Article or Practice).

3.2 Target users

User	How they use the system
Students	Enter their profile or select an existing student and receive a ranked list of topics to study next, with reasons.
Teachers and mentors	Review recommended topics and identify groups of students with similar needs through clustering.
E-learning platforms	Reuse the model and code as a starting point for a production recommendation service.
Students of machine learning	Study the complete workflow as a learning example.

3.3 Limitations

- The dataset is synthetic. It is generated with realistic rules (interest, difficulty fit, learning style and study hours influence interactions), so the reported metrics demonstrate that the pipeline works but do not prove real-world accuracy.
- The model is trained offline. It does not learn from new feedback in real time.
- Only 30 topics and 600 students are used, which keeps the system fast but small.
- Recommendations use ratings of 1 to 5 as the learning signal; richer signals such as quiz timing or video watch time are not used.
- Prerequisite relationships between topics are modelled only through difficulty level, not as a knowledge graph.

3.4 Assumptions

A rating of 3 or higher is treated as a positive interaction. A student's ability level is estimated from the previous score on a scale of 1 to 5. Students with at least three positive interactions are used for the hold-out evaluation.

4. Literature Review / Existing Systems

Recommender systems are a well-studied area. The main families of approaches, and how they apply to education, are summarised below.

Approach	Idea	Strength	Limitation
Popularity-based (common on e-learning platforms)	Recommend the items most students have used or rated highly.	Simple and works for everyone.	Same list for all students; no personalisation.
Content-based filtering (Lops et al.)	Match item features with the profile built from items a user liked.	Works without other users; explainable.	Over-specialisation; needs good item features.
Collaborative filtering (Resnick et al.; Sarwar et al.)	Recommend what similar users liked, using rating patterns.	Finds unexpected but relevant items.	Cold start and sparse data.
Matrix factorisation (Koren et al.)	Learn hidden factors for users and items from the rating matrix.	Accurate on large rating data.	Needs enough data; hard to explain.
Knowledge tracing (Corbett and Anderson)	Estimate how well a learner has mastered each concept over time.	Models learning progress directly.	Needs detailed step-level data.

4.1 Observations from existing work

- Learning data such as the Open University Learning Analytics Dataset (OULAD) shows that demographics, assessment results and click activity all influence outcomes, which supports using several student features together.
- Each single technique has a weakness: popularity ignores the individual, content-based filtering repeats the same kind of topic, and collaborative filtering fails for new users.
- Hybrid recommenders combine techniques to cover for these weaknesses, and are described in the Recommender Systems Handbook as a standard way to improve quality.

4.2 Gap addressed by this project

Existing e-learning recommenders are often generic, or they need large data and complex infrastructure. This project builds a small, transparent hybrid that adds learning-specific information (difficulty fit with the student's level, preferred resource type and interest) and handles new students through cluster-based popularity. Every recommendation is shown with a plain-language reason.

5. Proposed System

The proposed system is a hybrid recommender with four cooperating parts. Each part solves a specific weakness identified in the literature review.

Component	What it does	Problem it solves
Student profile clustering	K-Means groups students with similar scores, hours, attendance, style and interest into five clusters.	Cold start: a new student inherits the popular topics of the matching cluster.
Content-based scoring	Builds a taste profile from topics the student rated 3 or higher and scores topics by cosine similarity, then adjusts for difficulty fit and resource type.	Personalisation using the student's own history and level.
Collaborative filtering	Finds the 20 most similar students by cosine similarity on rating vectors and averages their ratings.	Discovers topics that similar students enjoyed.
Hybrid ranker	Combines the two scores as 0.6 x collaborative + 0.4 x content after min-max normalisation, hides topics already studied, and returns the Top-N.	Balances the strengths of both methods.

5.1 Key features

- Personalised Top-N recommendations for existing students and for brand-new students entering a profile.
- Explainable output: each recommendation shows reasons such as 'matches your interest in Databases' or 'difficulty suits your level'.
- Choice of method in the interface (hybrid, content, collaborative or popularity) to compare results.
- Built-in evaluation with Precision@K, Recall@K, NDCG@K and HitRate@K, and ready-made EDA charts.
- Fast, lightweight and fully reproducible: the dataset is generated automatically if it is missing.

5.2 Advantages over a popularity-only approach

A popularity list recommends the same topics to everyone. The proposed system recommends different topics to different students and, as the results in Section 13 show, places more of the topics a student actually went on to study within the top five suggestions.

6. Project Methodology

The project follows the standard machine learning workflow, with each phase implemented as a separate Python module so it can be tested and reused.

Phase	Activity	Module
1. Data collection	Generate a synthetic dataset of students, topics and interactions (or load real CSV files with the same columns).	src/data_gen.py
2. Cleaning	Remove duplicates, fill missing numeric values with the median and missing categories with the mode, clip scores to valid ranges, drop invalid interactions.	src/preprocess.py
3. EDA	Plot distributions, relationships and correlations; compute feature importance with a Random Forest.	src/eda.py
4. Feature engineering	Standardise numeric features and one-hot encode learning style and interest.	src/preprocess.py
5. Modelling	Fit K-Means, content-based scoring, user-based collaborative filtering and the hybrid ranker.	src/recommender.py
6. Evaluation	Hold out about 20 percent of each student's positive interactions and measure ranking quality.	src/evaluate.py
7. Deployment	Serve recommendations through a Streamlit application.	app.py

6.1 Tools and technologies

Category	Tool	Purpose
Programming language	Python 3	Entire implementation
Data handling	Pandas, NumPy	Tables, cleaning, matrix operations
Machine learning	Scikit-learn	K-Means, cosine similarity, scaling, encoding, Random Forest
Visualisation	Matplotlib	EDA and result charts
Interface	Streamlit	Interactive web application
Storage	CSV files	students.csv, topics.csv, interactions.csv
Testing and version control	Pytest, Git and GitHub	Unit tests, source control and sharing

7. System Architecture

The system is organised in four layers. Data flows upward from the data layer to the user interface, and each layer depends only on the one below it.

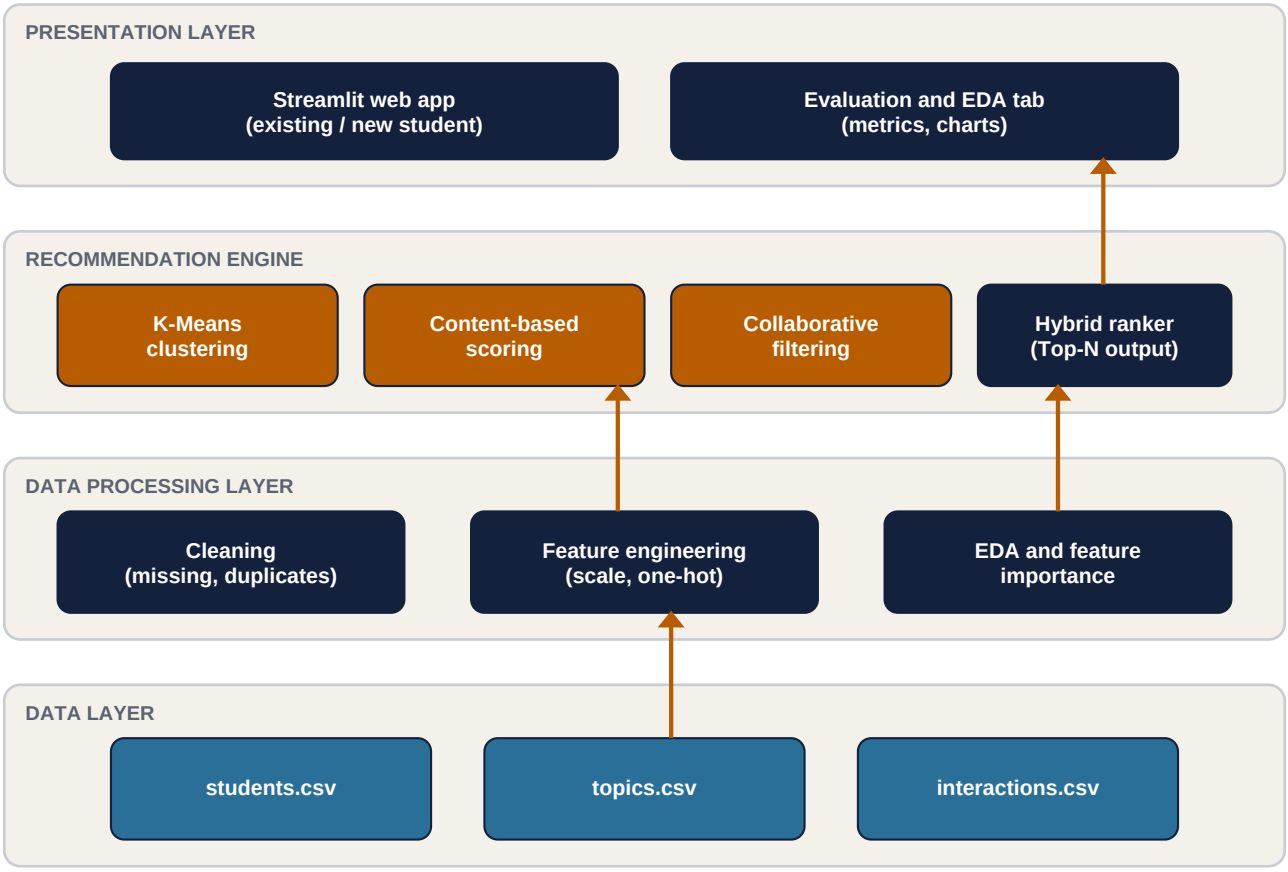


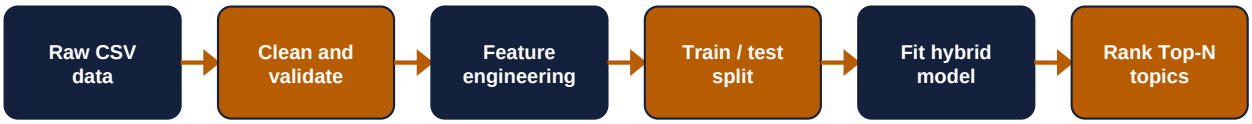
Figure 1: Layered architecture of the Personalized Learning Recommendation System

Description of the layers

- **Data layer:** three CSV tables hold students, topics and student-topic interactions.
- **Data processing layer:** cleaning removes errors, feature engineering prepares numeric inputs for clustering, and EDA reveals which features matter.
- **Recommendation engine:** K-Means, content-based scoring and collaborative filtering each produce scores, and the hybrid ranker merges them into a Top-N list.
- **Presentation layer:** the Streamlit app collects student information and displays recommendations, metrics and charts.

8. Database Design and Data Flow

8.1 Data flow



Outputs: ranking metrics, EDA plots and personalised recommendations with reasons

Figure 2: Data flow from raw data to recommendations

8.2 Database design

The data is stored in three related tables. **interactions** links students and topics (a many-to-many relationship) and stores the rating.

Table	Columns	Notes
students	student_id (key), prev_score, study_hours, attendance, learning_style, interest	600 rows. prev_score 0-100, attendance in percent.
topics	topic_id (key), topic, subject, difficulty, resource_type	30 rows. difficulty 1-5; resource Video, Article or Practice.
interactions	student_id (foreign key), topic_id (foreign key), score	4,771 rows. score is a rating 1-5; one row per student and topic.

8.3 Relationships

- One student has many interactions; one topic has many interactions.
- The student-topic rating matrix (600 x 30) is built from the interactions table and is used for similarity calculations.
- A topic is described by its subject, resource type and difficulty; these form the topic feature vector used by the content-based scorer.
- The same three-table structure can be moved to SQLite or any relational database without changing the model code.

9. Dataset and Preprocessing

9.1 Dataset description

A synthetic student-learning dataset was created with a fixed random seed (42) so that every run is identical. Interactions are generated from rules that mimic real learning behaviour: students are more likely to study and to rate highly the topics in their area of interest, topics whose difficulty is close to their ability, topics in their preferred resource type, and they study more when they have more hours available.

Item	Value
Students	600
Topics	30 (6 subjects x 5 difficulty levels)
Interactions	4,771 (about 8 per student, range 1 to 19)
Rating distribution (1 to 5)	104, 997, 2168, 1253, 249
Mean previous score	64.6 (standard deviation 14.5)
Mean study hours per week	5.9
Mean attendance	79.2 percent
Missing values before cleaning	14 in study_hours, 20 in attendance

9.2 Preprocessing steps

- **Duplicate removal:** repeated student records and repeated student-topic pairs are dropped.
- **Missing values:** numeric columns are filled with the median, categorical columns with the mode.
- **Range checking:** scores and attendance are clipped between 0 and 100; ratings outside 1 to 5 and interactions that point to unknown students or topics are removed.
- **Scaling:** prev_score, study_hours and attendance are standardised with StandardScaler so that no feature dominates K-Means.
- **Encoding:** learning_style and interest are one-hot encoded; unseen categories are ignored safely.
- **Ability estimate:** previous score is mapped to an ability level between 1 and 5, which is compared with topic difficulty.

9.3 Result of preprocessing

After cleaning, no missing values remain, and the feature matrix has 3 scaled numeric columns and 9 one-hot columns (3 learning styles and 6 interests), ready for clustering and for the feature importance model. This is checked by an automated unit test.

10. Exploratory Data Analysis

EDA was performed to understand the data and to decide which features should drive the recommendations. Four views are shown below.

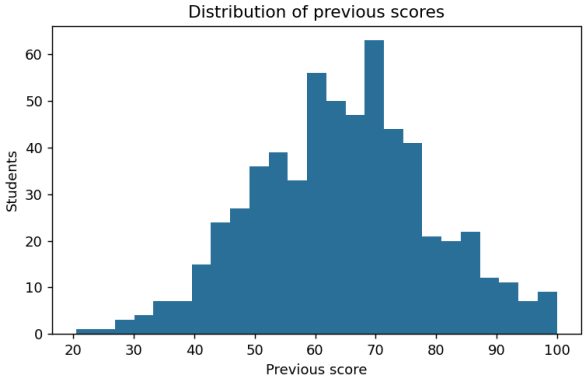


Figure 3: Distribution of previous scores

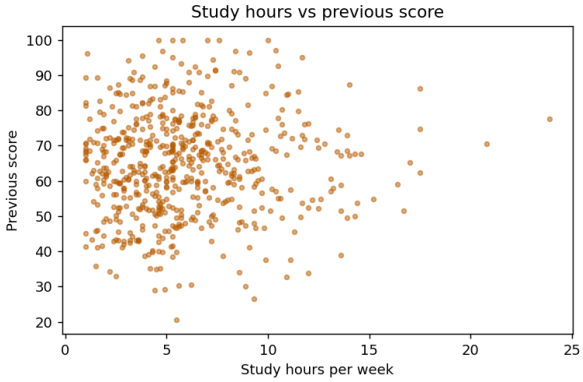


Figure 4: Study hours vs previous score

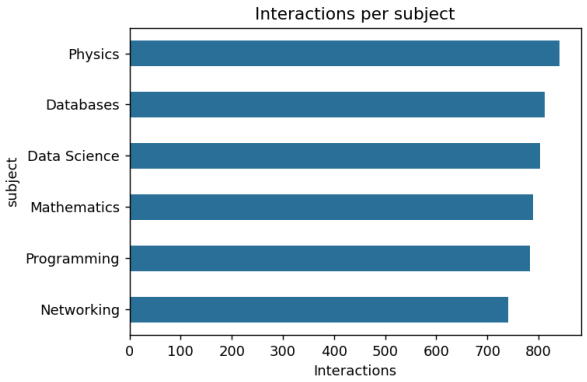


Figure 5: Interactions per subject

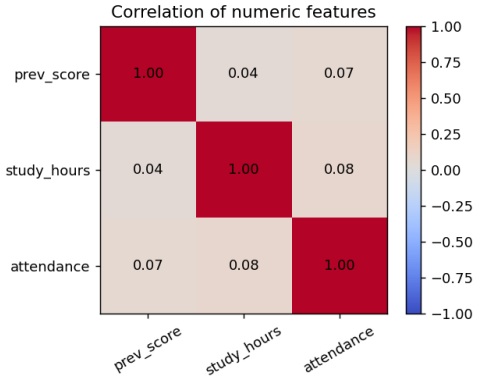


Figure 6: Correlation of numeric features

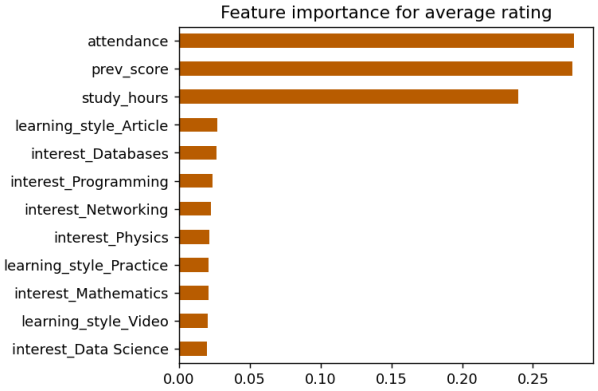
Findings

- Previous scores are roughly bell-shaped around 65, so students cover a wide range of ability levels and the recommender must adapt difficulty.
- Study hours and previous score are almost uncorrelated in this dataset, so both carry separate information about a student.
- Interactions are spread across all six subjects, which means no subject is ignored and the model has enough data for each.
- Most ratings are 3 or 4, with fewer very low and very high ratings, so a rating of 3 or more is a sensible threshold for a positive interaction.

11. Recommendation Model and Feature Importance

11.1 Important features

A Random Forest regressor was trained to predict each student's average rating from the profile features. Its importance scores show which features are most related to learning behaviour.



The three numeric features (attendance, previous score and study hours, about 0.24 to 0.28 each) carry most of the importance. Learning style and interest add smaller but useful signals and are mainly used by the content-based part to match topics.

Figure 7: Feature importance for average rating

11.2 Scoring formulas

```
Content profile      p_u   = sum( r_ui * f_i ) / sum( r_ui )    for topics with rating r_ui >= 3
Content score        CB_i  = cosine( p_u, f_i ) * exp(-|difficulty_i - ability_u| / 2) * (1 + 0.15 * style_match)
Collaborative score  CF_i  = sum_v( sim(u,v) * r_vi ) / sum_v( sim(u,v) )      over the 20 nearest students v
Similarity           sim(u,v) = (r_u . r_v) / ( ||r_u|| * ||r_v|| )
Hybrid score         S_i   = 0.6 * norm(CF_i) + 0.4 * norm(CB_i)     (min-max normalisation)
```

11.3 Cold-start handling

A new student has no ratings, so collaborative filtering cannot be used. The student's features are scaled and assigned to a K-Means cluster. The score is then 0.5 x the cluster's popularity for each topic plus 0.5 x a content score built from the student's interest, ability and learning style (and from any topics they report as completed).

11.4 Explanations

For every recommended topic the system lists the reasons that apply: it matches the student's interest, its difficulty is within one level of the student's ability, or its resource type fits the student's learning style. If none apply, the reason is that it is popular among similar students.

12. Implementation

12.1 Modules

File	Responsibility
data_gen.py	Creates topics, students (with a few missing values) and interactions; saves CSV files; loads them if they already exist.
preprocess.py	clean_students, clean_interactions, ability_level and the FeatureBuilder (scaling and one-hot encoding).
eda.py	Saves the EDA plots and computes feature importance.
recommender.py	HybridRecommender: fit, recommend, recommend_for_profile (new students) and top_ids (evaluation).
evaluate.py	Hold-out split and Precision@K, Recall@K, NDCG@K, HitRate@K.
run_pipeline.py / app.py	Command-line pipeline and the Streamlit web interface.

12.2 Core code (hybrid scoring in recommender.py)

```
def _scores(self, i, mode):
    if mode == "popularity": return self.pop.astype(float)
    if mode == "content":    return self._content(i)
    cf = self._cf(i)
    if mode == "collaborative": return cf
    if self.R[i].sum() == 0:   # cold start
        return 0.5*_norm(self.cluster_pop[self.cluster[i]]) + 0.5*_norm(self._content(i))
    return self.w_cf*_norm(cf) + self.w_cb*_norm(self._content(i))
```

12.3 User interface

The Streamlit app has two tabs. The **Recommendations** tab lets the user pick an existing student ID and a method, or enter a new student's previous score, weekly study hours, attendance, learning style, interest and completed topics. It then shows a ranked table of topics with scores and reasons, and a bar chart of the scores. The **Evaluation and EDA** tab shows the model comparison table and the EDA charts.

12.4 How to run

```
pip install -r requirements.txt     # install libraries
python run_pipeline.py              # generate data, EDA, evaluation
streamlit run app.py                # start the web app (http://localhost:8501)
pytest                              # run unit tests
```

13. Testing and Results

13.1 Test cases

Test	Checks	Result
test_cleaning_removes_missing	No missing values remain after cleaning.	Passed
test_recommend_excludes_seen_topics	Returns exactly N topics and none that the student has already studied.	Passed
test_new_student_profile	A brand-new student profile gets the requested number of recommendations.	Passed
test_ranking_metrics_perfect	Metrics return 1.0 recall, NDCG and hit when all relevant items are ranked first.	Passed

13.2 Model comparison

Each student's positive interactions (rating 3 or more) were split, with about 20 percent hidden for testing. Each method recommends five topics (K = 5) and is scored on the hidden topics.

Method	Precision@5	Recall@5	NDCG@5	HitRate@5
Popularity (baseline)	0.078	0.311	0.195	0.370
Content-based	0.116	0.461	0.321	0.534
Collaborative	0.108	0.438	0.321	0.501
Hybrid (proposed)	**0.126**	**0.502**	**0.361**	**0.568**

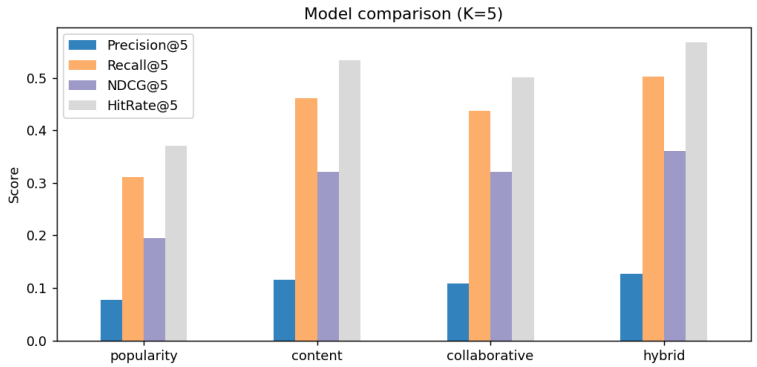


Figure 8: Comparison of the four methods at K = 5

13.3 Observations

- The hybrid model is best on all four metrics; its NDCG@5 of 0.361 is about 85 percent higher than the popularity baseline (0.195).
- About 57 percent of tested students had at least one hidden topic in their top five, against 37 percent for popularity.
- These numbers come from synthetic data and show the pipeline works; results on real data may differ.

14. Conclusion and Future Scope

14.1 Conclusion

The project designed, built and evaluated a Personalized Learning Recommendation System. A student-learning dataset was created and cleaned, EDA and feature importance showed which student features matter, and a hybrid recommender combining content-based filtering, collaborative filtering and K-Means clustering was developed. New students are served through cluster-based cold-start handling, and each recommendation is explained.

Evaluation against a popularity baseline showed that the hybrid approach gave the best Precision@5, Recall@5, NDCG@5 and HitRate@5. A Streamlit application and a GitHub-ready code repository make the system easy to run and reproduce.

14.2 How the objectives were met

Objective	Achieved through
Dataset	Synthetic dataset of 600 students, 30 topics and 4,771 interactions (data_gen.py).
Cleaning and preprocessing	Duplicate removal, imputation, range checks, scaling and encoding (preprocess.py).
EDA and important features	Four EDA plots and Random Forest feature importance (eda.py).
Recommendation model	Hybrid of content-based, collaborative and cluster-based scoring (recommender.py).
Personalised recommendations	Top-N output with reasons for existing and new students (app.py).
Evaluation and visualisation	Precision, Recall, NDCG and HitRate with comparison chart (evaluate.py, outputs/).

14.3 Future scope

- Train and test on a real dataset such as OULAD or data from a college learning management system.
- Add matrix factorisation or neural recommenders, and tune the hybrid weights automatically.
- Learn from real-time feedback and adapt difficulty as the student progresses.
- Model prerequisites between topics as a knowledge graph and use knowledge tracing.
- Use text features of resources (descriptions, transcripts) for richer content-based matching.
- Deploy as a web service with user accounts, a database, a teacher dashboard and a mobile app.

15. References and Appendix

References

[1] F. Ricci, L. Rokach and B. Shapira (eds.), *Recommender Systems Handbook*, Springer.

[2] Y. Koren, R. Bell and C. Volinsky, 'Matrix Factorization Techniques for Recommender Systems', *IEEE Computer*, 2009.

[3] P. Resnick et al., 'GroupLens: An Open Architecture for Collaborative Filtering of Netnews', *ACM CSCW*, 1994.

[4] B. Sarwar et al., 'Item-based Collaborative Filtering Recommendation Algorithms', *WWW Conference*, 2001.

[5] P. Lops, M. de Gemmis and G. Semeraro, 'Content-based Recommender Systems: State of the Art and Trends', in *Recommender Systems Handbook*, 2011.

[6] A. Corbett and J. Anderson, 'Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge', *User Modeling and User-Adapted Interaction*, 1994.

[7] J. Kuzilek, M. Hlosta and Z. Zdrahal, 'Open University Learning Analytics Dataset', *Scientific Data*, 2017.

[8] K. Jarvelin and J. Kekalainen, 'Cumulated Gain-based Evaluation of IR Techniques', *ACM Transactions on Information Systems*, 2002.

[9] J. MacQueen, 'Some Methods for Classification and Analysis of Multivariate Observations', *Berkeley Symposium*, 1967.

[10] L. Breiman, 'Random Forests', *Machine Learning*, 2001.

[11] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 2011.

[12] W. McKinney, 'Data Structures for Statistical Computing in Python', *SciPy Conference*, 2010; J. Hunter, 'Matplotlib: A 2D Graphics Environment', *Computing in Science and Engineering*, 2007.

[13] Streamlit documentation, https://docs.streamlit.io

Appendix: project repository structure

```
learning-recommender/
  app.py            Streamlit interface
  run_pipeline.py   data -> cleaning -> EDA -> evaluation
  src/              data_gen, preprocess, eda, recommender, evaluate
  tests/            pytest unit tests
  data/             students.csv, topics.csv, interactions.csv
  outputs/          metrics.csv, metrics.png, eda/*.png
  README.md  requirements.txt  LICENSE  .gitignore
```